ChatGPT

Database Systems (IS222P) – Exam Cheat Sheet

Lecture 8: Relational Algebra

- **SQL vs Relational Algebra:** SQL is a practical high-level query language; relational algebra (RA) is a theoretical, procedural query language defining the order of operations to obtain query results 【8†L4-L9】. Queries use RA operators to form expressions; RA underlies SQL and DBMSs (Oracle, SQL Server, etc.)【8†L4-L9】.
- **RA Operations:** Basic operations include **SELECT (σ)**, **PROJECT (π)**, **RENAME (ρ)** (unary operations) and **UNION (∪)**, **INTERSECTION (∩)**, **SET DIFFERENCE (−)**, **CARTESIAN PRODUCT (×)**, **JOIN (⋈)**, **DIVISION (÷)** (binary operations). RA operations form set-oriented algebra【8†L4-L9】.
- **SELECT (σ):** Chooses subset of tuples from a relation satisfying a condition【8†L6-L9】. Notation: σ<condition>(R). Properties: unary (applies to single relation), output degree = input degree, output tuples ≤ original (|σ_c(R)| ≤ |R|)【8†L9-L13】. Selection is commutative: σ_cond1(σ_cond2(R)) = σ_cond2(σ_cond1(R)). Consequence: SELECT operations can be cascaded in any order【8†L9-L13】.
- **PROJECT (π):** Selects specified columns (attributes) and discards others【8†L11-L14】. Notation: π<attr-list>(R). Attributes remain in the specified order; degree of result = number of attributes in list. Duplicate tuples are eliminated (duplicate elimination) – result is a set of distinct tuples【8†L12-L14】. The number of tuples in π(R) ≤ number in R【8†L13-L15】. Cascade: π_list1(π_list2(R)) = π_list1(R) if list2 contains list1 attributes【8†L13-L15】. Corresponding SQL uses `SELECT DISTINCT attr_list`【8†L13-L15】.
- **RENAME (ρ):** Renames relation name or attribute names (or both), unary. Notation:
- `ρ_S(R)` renames relation R to S.
- `ρ(B1,B2,…,Bn)(R)` renames attributes of R to B1…Bn.

- `ρ_S(B1,B2,…,Bn)(R)` renames both relation and attributes【8†L15-L16】. Use in SQL via `AS` (e.g., `SELECT Fname AS First_name, ... FROM EMPLOYEE AS E WHERE ...`)【8†L16-L17】.

- **Set Operators (∪, ∩, −):** Operate on union-compatible relations (same number of attributes, matching data types):

- **UNION (R ∪ S):** Tuples in R or S or both; duplicates eliminated【8†L18-L19】.
- **INTERSECTION (R ∩ S):** Tuples common to both R and S【8†L18-L19】.

- **DIFFERENCE (R − S):** Tuples in R but not in S【8†L18-L19】. Properties: ∪ and ∩ are commutative and associative (e.g., R∪S=S∪R; (R∪S)∪T=R∪(S∪T)), difference (−) is not commutative (R−S ≠ S−R generally)【8†L19-L20】. Intersection can be expressed via ∪ and −:
R ∩ S = ((R ∪ S) − (R − S)) − (S − R)【8†L19-L20】.

- **CARTESIAN PRODUCT (×):** Denoted R × S or CROSS JOIN【8†L20-L21】. Combines each tuple of R with each of S (binary operator; relations need *not* be union-compatible). Requires disjoint attribute names (no header overlap)【8†L20-L21】. Result attributes = all of R's then S's; degree = n+m if R has

n and S has m attributes【8†L20-L21】. Number of tuples = |R| × |S|【8†L20-L21】. Alone it is usually meaningless; it is typically followed by SELECT to match join conditions【8†L21-L24】.

- **CARTESIAN Product Example:** To list names of female employees' dependents:
- FEMALE_EMPS ← σSex='F'(EMPLOYEE)
- EMPNAMES ← πFname, Lname, Ssn(FEMALE_EMPS)
- EMP_DEPENDENTS ← EMPNAMES × DEPENDENT
- ACTUAL_DEPENDENTS ← σSsn=Essn(EMP_DEPENDENTS)

- RESULT ← πFname, Lname, Dependent_name(ACTUAL_DEPENDENTS)【8†L21-L24】.

- **JOIN (⋈):** Combines related tuples from two relations into single tuples【8†L22-L23】. A join can be expressed as a Cartesian product followed by a select on matching attributes【8†L22-L23】. Result has all attributes of both inputs (degree = n+m)【8†L23-L23】. A general join condition is a conjunction of comparisons (e.g., A.x = B.y AND ...). (Slide lacks full formula, but concept is as above.)

- **DIVISION (÷):** Denoted R ÷ S【8†L24-L25】. Given relations R and S, R ÷ S yields those tuples in R that are related to *all* tuples in S. (Formally, S's attributes must match some attributes of R.) It is the inverse of Cartesian product: if U = R × S, then U ÷ R = S and U ÷ S = R【8†L24-L25】. (Slide states U = R × S, then U ÷ R = S and U ÷ S = R, intuitive property【8†L24-L25】).
- **RA Example (Division-like query):** "Retrieve names of students with excellent grades in Database and age<20"【8†L26-L26】. Steps:
- STUDENTage ← σage<20(STUDENT)
- STUDENTname ← πName, ID(STUDENTage)
- DEGREEexcellent ← σSubject='Database' AND Grade>85(DEGREE)
- ExcellentStudent ← STUDENTname ⋈_{ID=StudID} DEGREEexcellent
- RESULT ← πName(ExcellentStudent)【8†L26-L26】.
- **Important Notes:** Every operator and rule above is as defined in the slides. Use symbols (σ, π, ρ, ∪, ∩, −, ×, ⋈, ÷) consistently. Steps and examples show how to express SQL queries in RA (as in examples above).

Lecture 9: Database Normalization (Part I)

- **Schema Quality Levels:** Two levels to evaluate relation schemas【9†L1-L4】:
- *Logical level:* How users interpret schemas and attribute meanings【9†L1-L4】.
- *Physical level:* How tuples are stored and updated in storage【9†L1-L4】.
- **Redundancy & Anomalies:** Storing redundant information in tuples wastes space and causes update anomalies【9†L5-L8】【9†L7-L11】:

- **Insertion Anomalies:**

1. Cannot insert an EMP_DEPT row without providing all department attributes or using NULL (violates consistency)【9†L8-L10】.
2. Cannot insert a new department (with no employees) into EMP_DEPT without NULLs in employee fields (violates PK constraint)【9†L9-L11】.

- **Deletion Anomalies:** Deleting the last employee of a department in EMP_DEPT loses that department's data【9†L11-L14】.

- **Modification Anomalies:** Changing a department attribute (e.g., manager) in EMP_DEPT requires updating all related tuples, else inconsistent values (department appears with different data in different tuples)【9†L12-L12】.
- **Normalization Definition:** Process of organizing the database to minimize redundancy and undesirable dependencies【9†L13-L14】. Involves dividing tables and defining relations to increase clarity and avoid data anomalies【9†L13-L14】. Achieved by following a set of rules (normal forms)【9†L14-L15】.
- **Functional Dependency (FD):** A constraint X → Y (X, Y subsets of R) means: for any two tuples t1, t2 in R, if t1[X] = t2[X] then t1[Y] = t2[Y]【9†L15-L17】. Y is functionally dependent on X. Notation: FD or f.d.; X is left side, Y is right side【9†L15-L17】.
- If X is a candidate key, then X → all attributes (X→R)【9†L17-L18】. If X→Y, it does *not* imply Y→X in general【9†L17-L18】.
- **Example FD:** In EMP_PROJ relation: Ssn→Ename; Pnumber→{Pname, Plocation}; {Ssn, Pnumber} →Hours【9†L18-L20】.
- **Normalization Goals:** Use FDs and keys to achieve: minimizing redundancy and insertion/deletion/modification anomalies【9†L19-L21】. If a schema fails a normal form test, decompose it into smaller schemas that meet the test【9†L19-L21】.
- **Normalization Procedure:** Provides a framework to analyze schemas using keys and FDs, and a series of normal-form tests【9†L20-L21】.
- **Normal Form Definition:** The highest normal form (1NF, 2NF, 3NF, BCNF, 4NF, 5NF) that a relation meets【9†L21-L23】. Achieve normalization by decomposing schemas to higher normal forms as needed.
- **Normal Forms List:** First Normal Form (1NF), Second NF (2NF), Third NF (3NF), Boyce-Codd NF (BCNF), Fourth NF (4NF), Fifth NF (5NF)【9†L22-L23】.

Lecture 10: Database Normalization (Part II)

- **First Normal Form (1NF):** A relation is in 1NF if:
- All attribute values are *atomic* (indivisible)【10†L1-L2】.
- No repeating groups or arrays (no duplicate columns)【10†L1-L2】.
- Once in 1NF, we can assign multiple values by separate rows instead of repeating fields【10†L5-L6】.
- **Second Normal Form (2NF):** Based on full functional dependency. Applies only when the primary key is composite (≥2 attributes)【10†L6-L10】. Conditions for 2NF:
- Relation is in 1NF.
- No partial dependency: i.e., no non-prime attribute is dependent on just part of a candidate key【10†L6-L10】.
- If any proper subset of a candidate key determines a non-key attribute, partial dependency exists (violating 2NF)【10†L7-L9】.
- If primary key is single attribute, relation is automatically in at least 2NF【10†L6-L10】.
- **2NF Example:** Relation R(STUD_NO, COURSE_NO, COURSE_FEE) with key {STUD_NO, COURSE_NO}. Since COURSE_NO → COURSE_FEE (COURSE_FEE dependent on part of key), not in 2NF【10†L9-L10】. To convert, decompose into:
- STUDENTS_COURSES(STUD_NO, COURSE_NO)

- COURSE_FEES(COURSE_NO, COURSE_FEE)【10†L10-L10】.
- **2NF Purpose:** Reduces redundant data (e.g., storing course fee only once per course instead of for each student)【10†L12-L12】.
- **2NF Quiz:** Given R(A,B,C,D) with FDs AB→C, BC→D, relation is in 2NF because no partial dependency (AB is key; no single part of AB determines a non-key)【10†L14-L14】.
- **Third Normal Form (3NF):** Relation is in 3NF if:
- It is in 1NF and 2NF.
- No non-key attribute is transitively dependent on the primary key (i.e., no A→B, B→C chain where C is non-key)【10†L15-L17】.
- Transitive dependency: if A→B and B→C, then A→C is transitive. Remove such dependencies by decomposition【10†L15-L17】.
- **3NF Example:** STUDENT(STUD_NO, Name, State, Age, Country) with FDs: Stud_NO→State, Stud_NO→Age, State→Country【10†L17-L18】. Stud_NO (key) transitively determines Country (Stud_NO→State→Country), violating 3NF【10†L17-L18】. Decompose into:
- STUDENT1(STUD_NO, Name, Phone, State, Age)
- STATE_COUNTRY(State, Country)【10†L17-L18】.
- **3NF Properties:** 3NF tables are free of insertion, update, deletion anomalies. 3NF ensures dependency preservation and lossless joins【10†L18-L19】.
- **Finding Highest NF:** Steps【10†L19-L20】:
- Identify all candidate keys.
- Classify attributes as prime (part of some key) vs non-prime.
- Check 1NF, then 2NF, etc., until a condition fails; highest NF is previous one【10†L19-L20】.

Lecture 11: Query Processing & Optimization

- **Query Processing:** Involves scanning, parsing, validating, optimizing, and executing a query 【11†L8-L11】. Steps:
- **Lexical analysis:** Scanner tokenizes the query.
- **Parsing:** Parser checks syntax.
- **Validation:** Ensure attribute/relation names exist and are semantically valid in the schema【11†L8-L11】.
- **Optimization:** Choose efficient execution strategy.
- **Query Blocks:** A single SELECT-FROM-WHERE (plus optional GROUP BY/HAVING) is a *query block* 【11†L9-L11】. Nested queries are separate blocks. Aggregates require extended algebra.
- **Query Tree:** A tree representation of an RA expression. Leaves = base relations; internal nodes = RA operations【11†L4-L6】【11†L13-L14】. Executing a query tree means performing each operation when inputs are ready, replacing node with result relation.

- **Translating SQL→RA Example:**
  SQL: `SELECT P.NUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE FROM PROJECT P, DEPARTMENT D, EMPLOYEE E WHERE P.DNUM=D.DNUMBER AND D.MGR_SSN=E.SSN AND P.PLOCATION='STAFFORD';`
  RA: π_NUMBER, DNUM, LNAME, ADDRESS, BDATE (((σ_PLOCATION='STAFFORD'(PROJECT)) ⋈*{DNUM=DNUMBER} DEPARTMENT)* ⋈ EMPLOYEE)【11†L15-L16】.

- **Heuristics (Query Optimization):** Use rules to simplify query trees and reduce cost【11†L13-L14】:

- **Push selection and projection down:** Apply σ and π as early as possible (close to leaves) to reduce intermediate sizes【11†L13-L14】.
- **Join vs Cartesian:** Replace Cartesian + select with join when possible.
- **Join order:** Reorder joins to apply most restrictive operations first【11†L13-L14】.
- **Transformation Rules (Relational Algebra):** Useful rewrite rules【11†L22-L26】:
- **σ cascade:** σ_(c1 AND c2 AND …)(R) = σ_c1(σ_c2(…(σ_cn(R))…)).
- **σ commutativity:** σ_c1(σ_c2(R)) = σ_c2(σ_c1(R)).
- **π cascade:** π_L1(π_L2(R)) = π_L1(R) if L1 ⊆ L2.
- **σ-π commute:** If σ condition only uses attributes in π list: π_L(σ_c(R)) = σ_c(π_L(R)).
- **Join/Cross commutative:** R ⋈ C S = S ⋈ C^R R; R × S = S × R (join and cartesian are commutative).
- **σ-join commute:** If σ condition involves only R attrs: σ_c(R ⋈ S) = (σ_c(R)) ⋈ S. More generally, split condition into c1 (R-only) and c2 (S-only): σ_(c1 AND c2)(R ⋈ S) = (σ_c1(R)) ⋈ (σ_c2(S)).
- **π-join commute:** If L = {A1..An from R, B1..Bm from S} and join condition uses only L attributes: π_L(R ⋈ S) = (π_A(R)) ⋈ (π_B(S)). (If join uses attrs not in L, include them in projection).
- **Set ops commutativity/associativity:** ∪, ∩ commutative; all four (⋈, ×, ∪, ∩) associative. (− is not commutative).
- **σ-set ops:** σ_c(R q S) = σ_c(R) q σ_c(S) for q ∈ {∪, ∩, −}.
- **π-union:** π_L(R ∪ S) = π_L(R) ∪ π_L(S).

- **σ×→⋈:** σ_C(R × S) = R ⋈_C S (if C is join condition).

- **Example (Heuristic Steps):** For a join query, one moves selections down the tree, applies the most selective σ first, replaces Cartesian+σ with joins, then pushes projections【11†L18-L19】. Figures (slides) illustrate these steps.

- **Practice Quiz:**
- (Example from slide) For schema *EMPLOYEE(FirstN,LastN,Address,Dept)* with Dept 'Research', write SQL, RA, and draw optimized tree (apply rules above).
